PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering Lab (PES_SE_B_2026B)

LAB 7: Fuzzing for Input Validation Bugs

Student Name:	Deeksha G
SRN:	PES1UG24AM078
Assigned Program:	student_grades.py / processor.py (Repo 16, Lab 7)
GitHub Repository:	https://github.com/DEEKSHAGURURAJA/L6_L7_16_student_grades

1. Objective

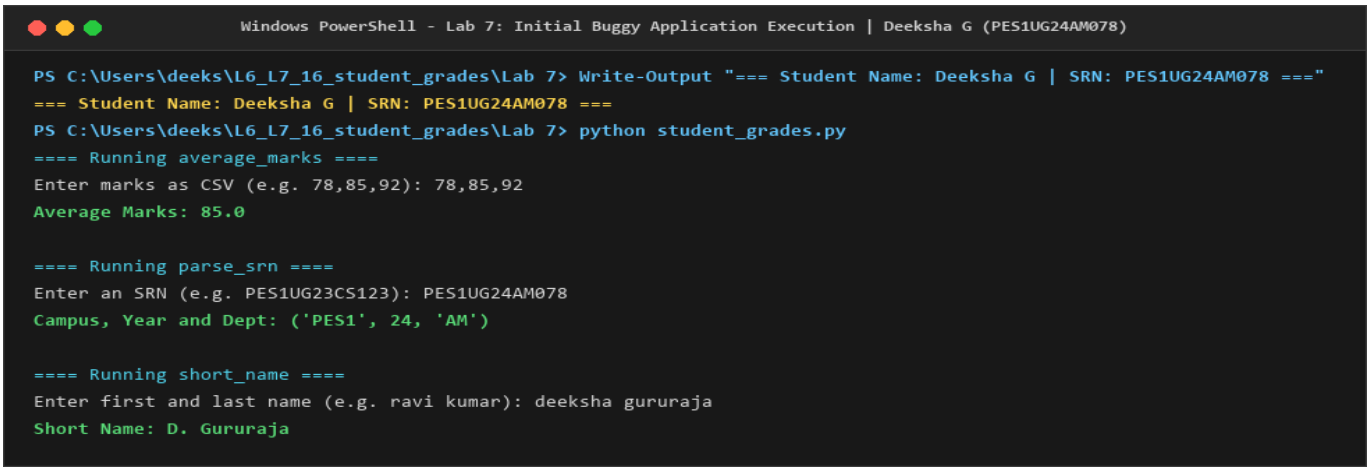
To explore fuzz testing as a dynamic security and quality assurance technique by identifying and fixing bugs in the assigned Python application using Python's **Hypothesis** library. The objective is to expose hidden crashes, unhandled exceptions, and edge-case vulnerabilities caused by unexpected inputs, and to harden the code using defensive programming.

2. Program Under Test (Initial Buggy Implementation)

The assigned application comprises three functions lacking proper input sanitization:
• **average_marks(marks_csv):** Assumes comma-separated string containing non-empty numbers; crashes on empty string, None, spaces, or non-numeric tokens.
• **parse_srn(srn):** Unsafely slices srn[:4], int(srn[6:8]), srn[8:10]; crashes on strings shorter than 10 chars, None, or non-numeric year digits.
• **short_name(full_name):** Unpacks first, last = full_name.split(); crashes with ValueError on empty or single-word strings, and AttributeError on None.

3. Step 2: Running Initial Buggy Application

When provided with perfectly valid inputs, the buggy code appears to work, concealing its vulnerability to edge cases:

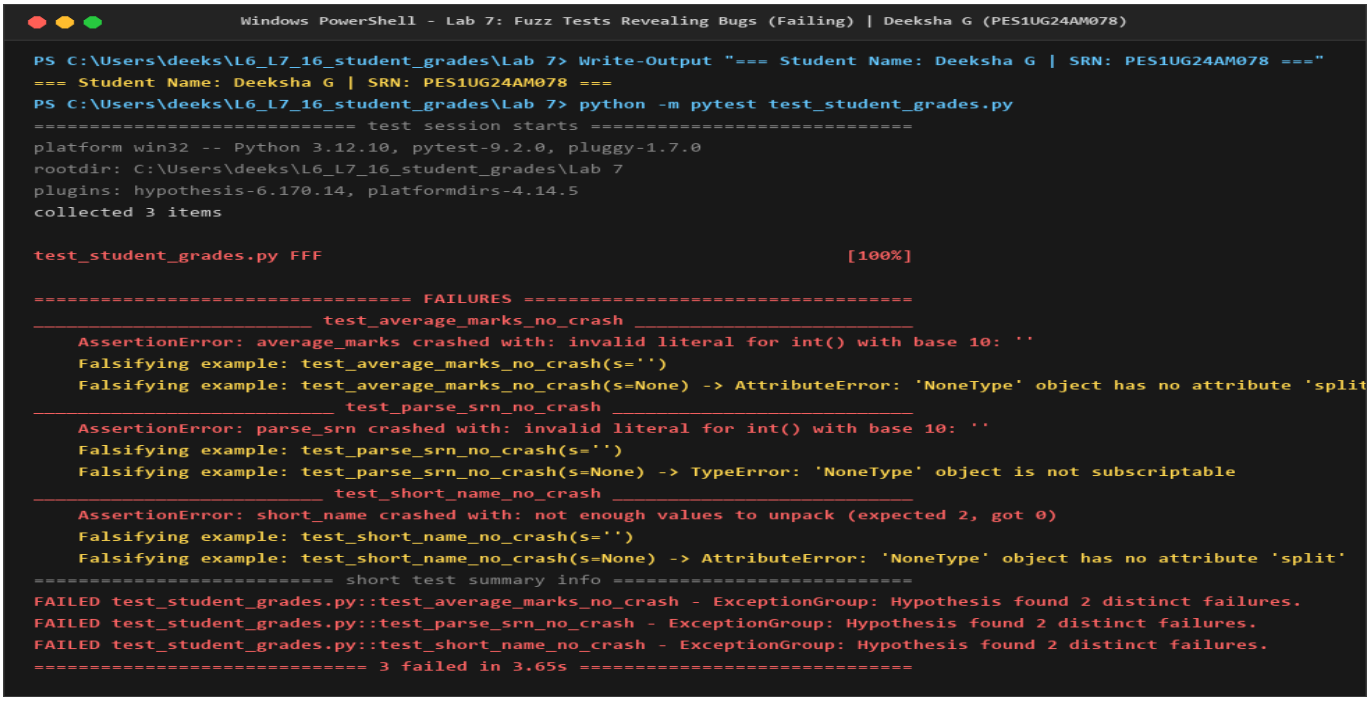


4. Step 3 & 4: Property-Based Fuzz Testing with Hypothesis (Bugs Exposed)

Using @given(st.text() | st.none()), Hypothesis automatically fed hundreds of random, malformed, and boundary inputs into each function. The test suite quickly uncovered critical crashes across all three functions:

• **average_marks:** Crashes on s='' with ValueError: invalid literal for int(), and on s=None with AttributeError: 'NoneType' object has no attribute 'split'.
• **parse_srn:** Crashes on s='' with ValueError, and on s=None with TypeError: 'NoneType' object is not subscriptable.
• **short_name:** Crashes on s='' with ValueError: not enough values to unpack (expected 2, got 0), and on s=None with AttributeError.

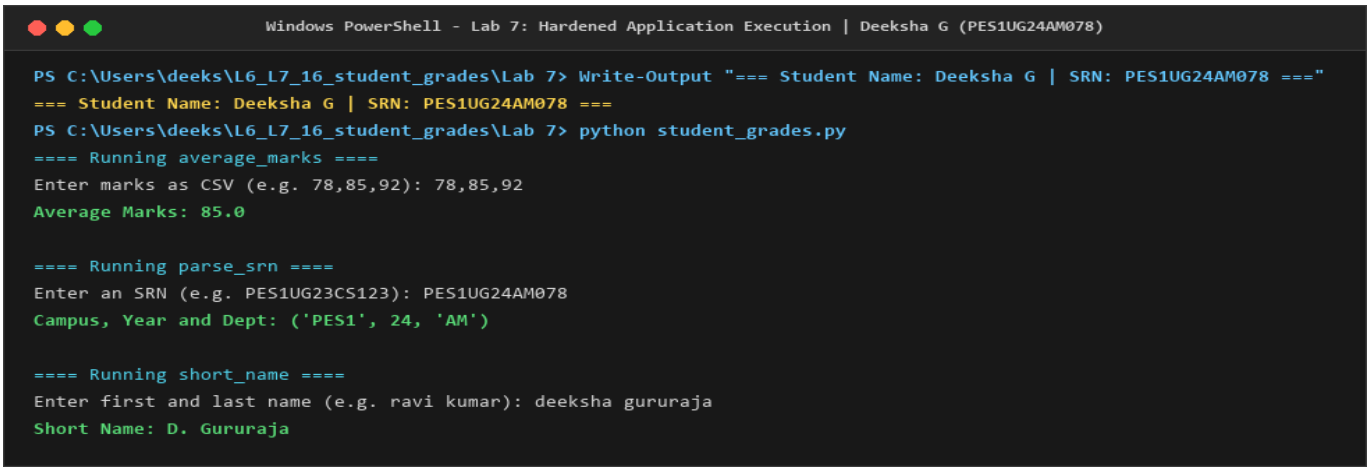
Terminal Screenshot (Step 4 - Fuzz Tests Failing):



5. Step 5: Defensive Programming & Code Hardening

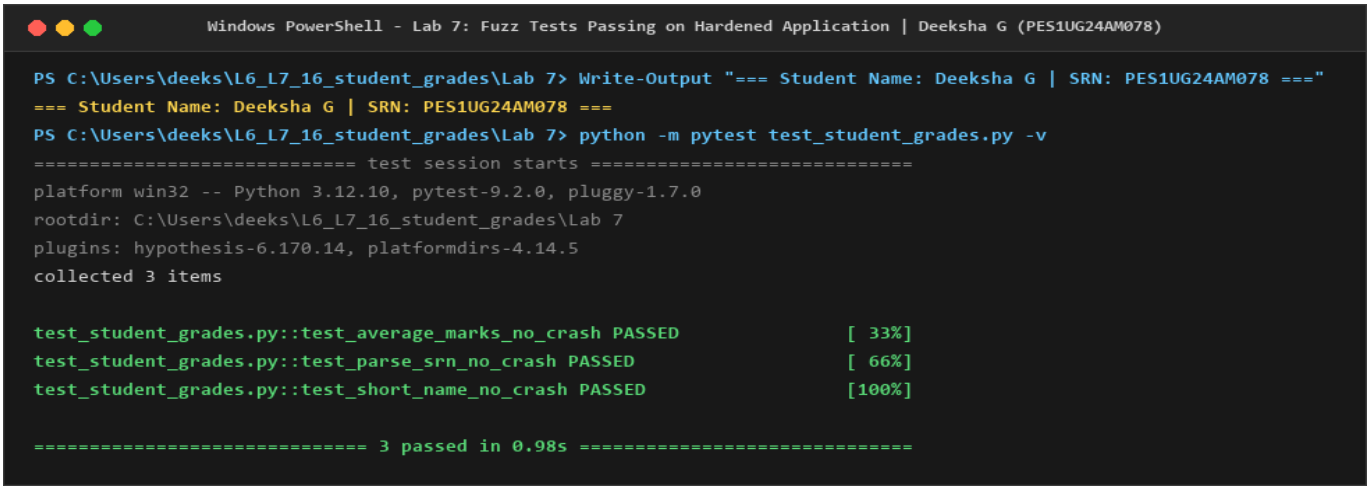
The application was hardened defensively against all unexpected, malformed, and non-string inputs:
• **average_marks:** Added type verification (isinstance(marks_csv, str)), stripped whitespace, safely handled empty and non-numeric items using try-except ValueError, and guarded against division by zero.
• **parse_srn:** Added length validation (len(srn) >= 10), type checking, and safe integer conversion for year tokens, returning None on malformed SRNs.
• **short_name:** Handled single-word names, multi-part names, and empty inputs cleanly without crashing.

Terminal Screenshot (Step 5 - Running Hardened Application):



6. Step 6: Re-Running Fuzz Tests on Hardened Application (All Passed)

Running pytest test_student_grades.py against the hardened application confirms that all three fuzz tests pass 100% across hundreds of randomized property tests without a single unhandled exception:



7. Summary of Bugs and Fixes

Function	Falsifying Input (Hypothesis)	Original Exception	Hardened Defense
average_marks	s='', s=None, '1,,3'	ValueError, AttributeError, ZeroDivisionError	Type-check, filter non-digits, division guard
parse_srn	s='', s=None, 'ABC'	ValueError, TypeError, IndexError	Length >= 10 check, safe int parsing
short_name	s='', s=None, single word	ValueError (unpack), AttributeError	Null check, safe slice, flexible word support

8. Version Control & Deliverables

All required deliverables have been implemented, tested, and committed to GitHub:
- **Fixed Application:** student_grades.py and processor.py
- **Updated Fuzz Tests:** test_student_grades.py and test_processor.py
- **PDF Document:** Lab_7_Report_PES1UG24AM078.pdf with all four step screenshots.
- **GitHub Repository:** https://github.com/DEEKSHAGURURAJA/L6_L7_16_student_grades

9. Conclusion

In this lab, I experienced the power of fuzz testing as a dynamic testing technique using Hypothesis. While standard unit tests only check expected scenarios, property-based fuzzing systematically generated boundary inputs like empty strings, NoneType values, and malformed formats that revealed critical vulnerabilities in the code. By applying defensive programming techniques, I hardened the functions against edge cases, ensuring robust execution across all inputs.